Tree Traversal (CodeGrade)

 (https://github.com/learn-co-curriculum/python-p3-dsa-tree-traversal)  (https://github.com/learn-co-curriculum/python-p3-dsa-tree-traversal/issues/new)

Learning Goals

- Understand what tree traversal is
- Understand breadth-first vs. depth-first traversal
- Implement breadth-first and depth-first traversal algorithms
- Discuss Big O considerations of tree traversal algorithms
- Use tree traversal to recreate the `getElementById` method

Introduction

In the last lesson, we learned about different types of trees, how they're constructed, and what they're used for. In this lesson, we'll learn two different methods for visiting the nodes in a tree.

Types of Tree Traversal

Tree traversal refers to the process of visiting each of the nodes in a tree in some systematic way. There are several approaches that are commonly used, each of which results in the nodes being visited in a particular order. These approaches can be divided into two general categories: **breadth-first** and **depth-first**.

Breadth First

In breadth-first traversal, we start at the root, then visit the remaining nodes level by level, left to right:

In this example, the nodes would be visited in the following order:

```
20 -> 50 -> 2 -> 11 -> 45 -> 8 -> 101 -> 39 -> 75 -> 57
```

Say we want to build a method on our `Tree` class that takes a node as input and returns an list containing the values of the root node and all of its child nodes in breadth-first order. How would we go about building this in code? We will need to use a second list to keep track of which nodes we still need to visit. The order in which we add nodes to this second list will control the order of the elements in the output list.

Let's start by writing some pseudocode:

```
Initialize an empty output list
Initialize an list of nodes to visit and add the root node to it
While there are nodes in the nodes to visit list
  Remove the first node from the nodes to visit list
  Add its value to the output list
  Add its children (if any) to the end of the nodes to visit list
Return the output list
```

Take a couple of minutes to walk through the pseudocode using the example tree in the diagram above so you can visualize how it works.

Let's start by defining our method and creating our `nodes_to_visit` and `result` list variables. We also want to set up our method to take a node as an argument, and initialize `nodes_to_visit` with that variable:

```
def breadth_first_traversal(node):
  result = []
  nodes_to_visit = [node]
```

Next, we'll create our `while` loop:

```
def breadth_first_traversal(node):
  result = []
  nodes_to_visit = [node]

  while len(nodes_to_visit) > 0:

    # traverse our node
```

Inside our `while` loop, we want to do three things:

1. Remove the first node from the `nodes_to_visit` list
2. Add its value to the result list, and
3. Add its children (if any) to the `nodes_to_visit` list

```
def breadth_first_traversal(node):
  result = []
  nodes_to_visit = [node]

  while len(nodes_to_visit) > 0:
    # 1. Remove the first node from the `nodes_to_visit` list
    node = nodes_to_visit.pop(0)
    # 2. Add its value to the result list
    result.append(node['value'])
    # 3. Add its children (if any) to the END of the `nodes_to_visit` list
    nodes_to_visit = nodes_to_visit + node['children']

  return result
```

Let's call our method using the following very simple node as input:

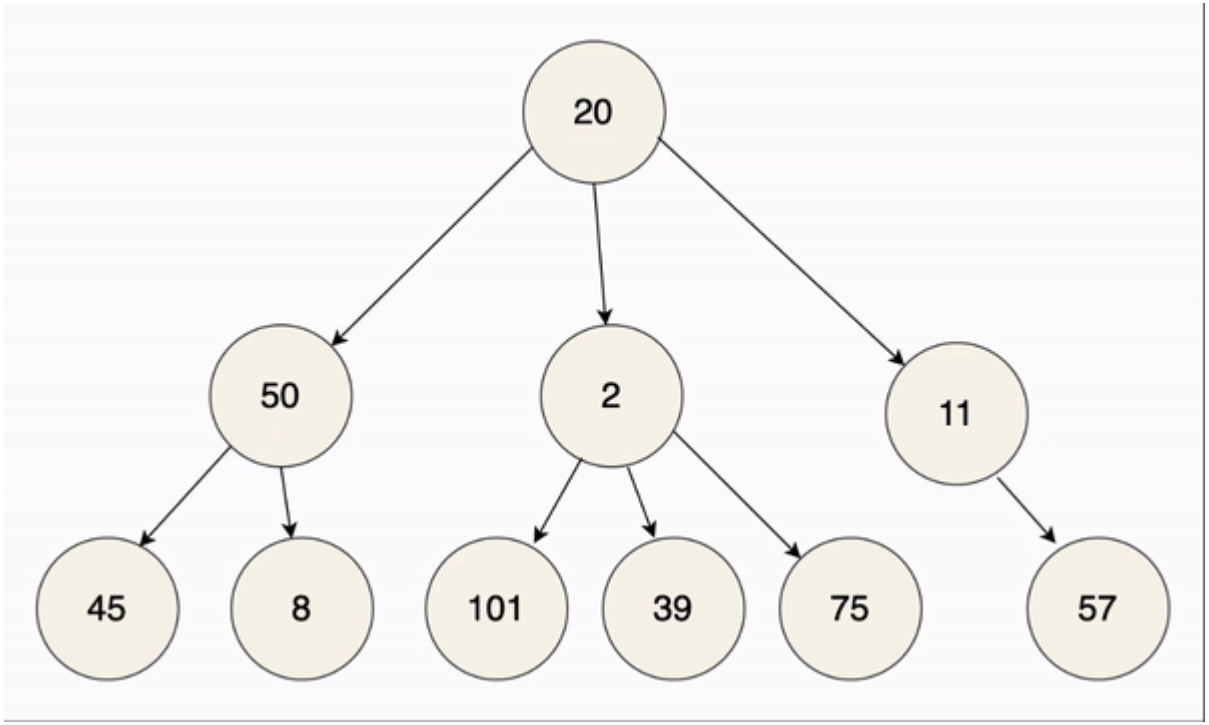
We set up our variables, then call the method, passing the root node as an argument:

```
child_1 = {
  'value': 2,
  'children': []
}

child_2 = {
  'value': 3,
  'children': []
}

child_3 = {
  'value': 4,
```

```
  'children': []
}

root = {
  'value': 1,
  'children': [child_1, child_2, child_3]
}
print(breadth_first_traversal(root))
# => [1, 2, 3, 4]
```

Depth First

With depth-first traversal, rather than visiting nodes level by level, we instead work our way down to the bottom of the tree first. Once we get to the bottom, we backtrack until we get to a node that hasn't been fully explored yet, work our way down to the bottom again, and so on until we're done:



This method of depth-first traversal is also known as **preorder traversal**. Once we're done, we will have visited the nodes in the following order:

```
20 -> 50 -> 45 -> 8 -> 2 -> 101 -> 39 -> 75 -> 11 -> 57
```

So how would we go about building this in code? Well the good news is that the process is almost identical to the breadth-first traversal!

Let's think about what we did in that case. We started at the root (20), then visited its left-most child (50). We added that node's children to the end of the list of nodes to visit, then continued visiting the remaining children of the root node (2 and 11). In this case, however, we want to visit the children of 50 **before** we visit its siblings. Doing that is just a matter of making one small change to our earlier code.

Here's what our pseudocode would look like:

```
Initialize an empty output list
Initialize an list of nodes to visit and add the root node to it
While there are nodes in the list of nodes to visit
  Remove the first node from the list of nodes to visit
  Add its value to the output list
  Add its children (if any) to the BEGINNING of the list of nodes to visit
Return the output list
```

Once again, spend a couple of minutes working through the process with our example tree to see how it works.

The final code looks like this:

```
def depth_first_traversal(node):
  result = []
  nodes_to_visit = [node]

  while len(nodes_to_visit) > 0:
    # 1. Remove the first node from the `nodes_to_visit` list
    node = nodes_to_visit.pop(0)
    # 2. Add its value to the result list
```

```
    result.append(node['value'])
    # 3. Add its children (if any) to the BEGINNING of the `nodes_to_visit` list
    nodes_to_visit = node['children'] + nodes_to_visit

  return result
```

Note that the only change in our method was to add the child nodes to the **beginning** of the `nodes_to_visit` list instead of the end.

Depth-first search also lends itself well to a recursive solution, where we traverse each sub-tree of the node's children recursively before moving to the next sub-tree:

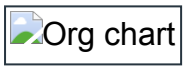
```
def depth_first_traversal(node, result = []):
  # visit each node (add it to the list of results)
  result.append(node['value'])
  for child in node['children']:
    # visit each child node
    depth_first_traversal(child, result)

  return result
```

When to Use Breadth-First vs. Depth-First Traversal Methods

Unfortunately, there are no hard and fast rules here — it really depends on the circumstances. However, there are a couple of factors that can provide some guidance.

The Type of Output Needed

In some cases, the desired output will determine what method of traversal should be used. Say, for example, we have a company organization chart that lists all the employees hierarchically:



On occasion, we may need to print out all the employees. If we want to print them in hierarchical order — the CEO first, followed by Vice Presidents, Managers, etc. — we would use a breadth-first traversal. But if we want to print out the employees by function or department — the head of the department or function first, followed by all their direct reports, then all the employees who report to *them*, etc. — we would use a depth-first approach.

Big O Considerations

Setting aside binary search trees — which, as we discussed in the previous lesson, are usually (but not always) more efficient — the time complexity of traversing trees is the same regardless of the approach used. No matter which way we proceed through the tree, we need to visit every node, giving a time complexity of O(n). Therefore, time complexity will not help us pick one approach over the other.

Space complexity considerations, however, *can* help us choose. Because we use an additional data structure to hold elements as we go (as we did in the examples above with the holder list), we want to choose a method that minimizes the storage requirements. To do this, we need to consider the **height** vs. the **width** of our tree.

With breadth-first traversal, we work our way across each level, adding the children of each node to our holder list as we go. With depth-first traversal, on the other hand, we work our way down from each child node in turn, adding children of that node at each successive level. As a result, if we have a very wide tree, where there are not a lot of levels but there are many nodes on each level, the storage requirements will be less if we use depth-first search. Conversely, if we have a long skinny tree, where each node has many children and grandchildren, but there aren't a lot of nodes within each level, breadth-first traversal will be more efficient.

Exercise: Build `getElementById`

Let's get some practice using our traversal skills by creating a Python version of JavaScript's `Document.getElementById` **method** (https://developer.mozilla.org/en-US/docs/Web/API/Document/getElementById).

In the `lib` folder, we having included an implementation of a `Tree` class. The nodes in the `Tree` will be structured as follows:

```
{
  'tag_name': 'h1',
  'id': 'heading',
```

```
    'text_content': 'Title',
    'children': []
}
```

You do not need to create nodes or a `Node` class yourself — the tests will handle that part.

To pass the tests, you will need to add an instance method, `get_element_by_id`, to the `Tree` class that uses the depth-first approach to traverse the `Tree` and find the desired node. Like the browser's `getElementById` method, your method should take a string as an argument and return the node with that value. If a matching node is not found, your method should return `None`.

Once you have the tests passing, try modifying your method to use breadth-first traversal instead; the tests should still pass.

Resources

- **BaseCS: Tree Traversal (https://www.youtube.com/watch?v=eFK3pzJEHKI)**

(https://www.youtube.com/watch?v=eFK3pzJEHKI)

- **Tree Traversal Space Requirements (https://eugene-eeo.github.io/blog/tree-traversal-storage.html)**

This tool needs to be loaded in a new browser window

Load Tree Traversal (CodeGrade) in a new window